

DÉVELOPPEMENT D'APPLICATIONS MOBILES

Projet en évaluation continue et défense orale

Année académique	2026-2027
Enseignant	DYLAN JACQUET
Programme	B3 · Bachelier en informatique

Répartition de la note finale
Projet en évaluation continue : 30 %
Défense orale individuelle : 70 %

Version destinée aux étudiants

1. Objectifs de l'évaluation

L'évaluation vérifie la capacité de l'étudiant à concevoir une application mobile pertinente avec Flutter, à appliquer les concepts étudiés au cours et à maîtriser les choix techniques réalisés dans son propre projet.

Le projet permet de construire ces compétences progressivement. La défense orale vérifie ensuite la compréhension personnelle du travail, la capacité à justifier les décisions prises et la maîtrise des notions techniques mobilisées.

Composante	Modalité	Pondération
Projet	Évaluation continue pendant l'année	30 %
Oral	Défense individuelle du projet	70 %

Calcul de la note finale — moyenne géométrique pondérée
Si P est la note du projet sur 20 et O la note de la défense orale sur 20 :
$$\text{Note finale} = P^{0,30} \times O^{0,70}$$

Si l'une des deux notes vaut 0, la note finale vaut également 0.

2. Projet — évaluation continue (30 %)

L'étudiant développe une application mobile personnelle qui répond à un besoin identifiable pour un utilisateur final. Le choix du besoin reste libre, sous réserve de la validation du sujet et de son périmètre par l'enseignant.

Le projet doit être suffisamment conséquent pour démontrer une maîtrise réelle des apprentissages. Un CRUD complet constitue uniquement le minimum technique requis. Il ne suffit pas, à lui seul, à garantir la réussite du projet.

L'application doit former un ensemble cohérent, utile et fonctionnel. Elle doit répondre à un besoin réel ou vraisemblable, proposer un parcours utilisateur qui a du sens et intégrer plusieurs fonctionnalités qui servent ce besoin.

Une application limitée à un CRUD générique, à une simple démonstration technique ou à une juxtaposition d'écrans sans finalité sera considérée comme insuffisante. Une application jugée insuffisante peut entraîner une note inférieure à 10/20 pour la partie projet, même si certains éléments minimaux sont présents.

2.1. Fonctionnalités minimales

- construire une interface cohérente et assurer la navigation entre plusieurs écrans ;
- afficher une collection de données et consulter le détail d'un élément ;
- implémenter un CRUD complet : création, consultation, modification et suppression d'au moins une ressource ;
- saisir et valider des données au moyen de formulaires adaptés ;
- gérer les états de chargement, les données absentes, les erreurs et les saisies incorrectes ;
- adapter les fonctionnalités et l'expérience utilisateur au besoin choisi et au public cible.

L'authentification, l'utilisation d'une fonctionnalité propre à l'appareil, les notifications, les animations ou la ludification peuvent compléter le projet lorsqu'elles servent réellement le besoin. Leur présence ne compense pas un manque de cohérence ou de profondeur fonctionnelle.

2.2. Concepts vus en classe

Le projet doit mobiliser les concepts, outils et pratiques abordés en classe. L'étudiant doit les adapter à son application, les intégrer correctement et pouvoir justifier ses choix. Une solution qui contourne les concepts imposés ne répond pas complètement aux objectifs de l'évaluation.

- respecter l'architecture enseignée, notamment la séparation en trois projets distincts, et répartir clairement les responsabilités ;
- factoriser les widgets, limiter le couplage et séparer les considérations graphiques de la logique applicative ;
- utiliser des modèles métier typés, des DTO et les mécanismes de conversion ou de mapping enseignés ;
- gérer les données asynchrones et les flux au moyen des mécanismes étudiés, par exemple StreamBuilder ou les builders adaptés à Firebase ;
- maintenir un code lisible, cohérent et sans avertissement ni erreur dans l'analyse Dart.

2.3. Persistance obligatoire avec une base NoSQL

Le projet doit obligatoirement utiliser un service reposant sur une base de données NoSQL, tel que Firebase avec Cloud Firestore. Une solution équivalente ne peut être utilisée qu'avec l'accord préalable de l'enseignant et si elle permet d'évaluer les mêmes apprentissages.

- organiser les collections et les documents en fonction des besoins de lecture et d'écriture ;
- choisir et justifier l'imbrication, les références et la dénormalisation des données ;
- éviter de transposer sans adaptation un schéma relationnel dans une base NoSQL ;
- effectuer les accès aux données par l'intermédiaire d'objets métier typés ;
- prendre en charge les mises à jour en temps réel et le comportement hors connexion lorsque la technologie le permet.

API personnalisée interdite. L'étudiant ne peut pas développer ni utiliser une API REST, GraphQL ou tout autre backend personnalisé créé par lui pour servir de couche d'accès aux données. L'application doit utiliser directement les SDK et mécanismes d'intégration étudiés au cours.

2.4. Documentation et dépôt GitHub

Le projet est déposé dès le début sur GitHub, dans l'organisation officielle de l'année en cours, au moyen du lien publié sur Moodle. Des commits réguliers et explicites doivent permettre de suivre la progression réelle du travail.

Le dépôt GitHub doit contenir au minimum :

- l'intégralité du code source des trois projets constituant l'architecture demandée ;
- les fichiers de configuration, pubspec.yaml, dépendances et ressources nécessaires à la compilation ;
- les modèles métier, DTO, convertisseurs, configurations Firebase et autres fichiers techniques nécessaires au fonctionnement ;
- les maquettes, wireframes, moodboards, inspirations et documents de conception produits ;
- les récits utilisateurs, préparations, analyses et livrables intermédiaires demandés pendant le cours ;
- les images, icônes, polices, traductions, données de démonstration et autres ressources utilisées par l'application ;
- les tests éventuels et toute donnée ou instruction nécessaire pour reproduire une démonstration ;
- un fichier d'exemple pour les configurations qui ne peuvent pas être publiées, sans déposer de secret ou de clé privée ;
- un fichier .gitignore adapté, excluant les fichiers générés, les dossiers de compilation et les informations sensibles ;
- un fichier README.md complet et régulièrement mis à jour.

Le fichier README.md comprend au minimum :

- une présentation du besoin, du public cible et de la manière dont l'application y répond ;
- une présentation des fonctionnalités sous la forme de récits utilisateurs, accompagnés des maquettes pertinentes ;
- un état d'avancement régulièrement mis à jour et des illustrations des fonctionnalités terminées ;
- une brève étude de l'existant avec les forces et faiblesses des solutions comparables ;
- une présentation de l'organisation du dépôt et de l'architecture du code ;
- les étapes nécessaires pour compiler et exécuter l'application, les versions requises, les dépendances et les éventuelles clés API ;
- les limites connues ainsi que les sources externes, extraits de code et aides utilisées, y compris l'intelligence artificielle.

2.5. Maquettes et expérience utilisateur

Les étudiants en infographie doivent produire leurs maquettes avant l'implémentation et renseigner clairement leur moodboard. Pour les autres étudiants, les maquettes restent recommandées et doivent être fournies lorsqu'elles sont demandées pendant le suivi.

Les choix de conception doivent tenir compte du public cible, des petits écrans et des principes de conception inclusive. L'interface doit rester utilisable sans débordement sur la taille d'écran prévue.

2.6. Suivi, préparations et étapes évaluées

Le projet est évalué tout au long du cours. Le suivi régulier, la progression entre les séances ainsi que la qualité et la ponctualité des préparations et livrables intermédiaires demandés font partie de l'évaluation continue. Les dates précises sont communiquées sur Moodle.

Une présentation du concept de l'application devant la classe est organisée au début du développement. Cette présentation est évaluée. L'étudiant y expose le besoin, le public cible, la proposition de valeur, les fonctionnalités principales et les premières orientations techniques et visuelles.

Étape	Éléments observés
1. Cadrage et préparations	Besoin, public cible, fonctionnalités, analyse de l'existant, modèle de données et premières maquettes.
2. Présentation du concept	Présentation évaluée devant la classe : pertinence, clarté, cohérence et capacité à répondre aux premières questions.
3. Version intermédiaire	Premier parcours fonctionnel, intégration NoSQL, architecture, progression et prise en compte des retours.
4. Remise finale	Application complète, cohérente, compilable et documentée, tenant compte des retours.

La note du projet prend en compte les préparations demandées, la présentation du concept, la régularité du travail, la progression observée, l'utilisation des séances de suivi, la prise en compte des conseils, la pertinence fonctionnelle, la qualité technique et la documentation. La compréhension personnelle et la capacité à défendre les choix sont évaluées principalement lors de la défense orale finale.

3. Défense orale individuelle (70 %)

La défense orale porte directement sur le projet remis. Elle permet à l'étudiant de présenter son application, de justifier ses décisions et de démontrer qu'il comprend personnellement l'ensemble du travail réalisé.

L'enseignant pose des questions ciblées sur le projet et peut choisir librement les écrans, fichiers, classes, modèles, flux de données ou fonctionnalités que l'étudiant doit expliquer.

Principe essentiel. Une application peut sembler réussie et fonctionner correctement sans que son auteur maîtrise réellement les choix effectués. La qualité apparente du résultat ne suffit donc pas à garantir la réussite de l'évaluation orale.

3.1. Contenu de la défense

L'étudiant doit notamment être capable de :

- présenter le besoin traité, le public cible et les principales fonctionnalités ;
- effectuer une démonstration structurée de l'application ;
- justifier l'architecture, l'organisation des fichiers et la séparation des responsabilités ;
- expliquer la modélisation NoSQL, les collections, les documents, les références et les choix de dénormalisation ;
- expliquer le rôle des modèles métier, des DTO, de l'ODM et des convertisseurs typés ;
- décrire la gestion de l'état, des flux asynchrones, du chargement et des erreurs ;
- expliquer les choix d'interface, de navigation et d'expérience utilisateur ;
- identifier les limites du projet et proposer des améliorations réalistes.

3.2. Questions techniques spécifiques

Les questions ne se limitent pas à une présentation préparée. Elles servent à vérifier les connaissances techniques à partir de situations concrètes présentes dans le projet. L'étudiant peut devoir expliquer le fonctionnement d'un extrait de code, suivre le trajet d'une donnée, anticiper l'effet d'une modification, repérer une faiblesse ou proposer une correction.

Les questions peuvent notamment porter sur Flutter, Dart, les widgets, la navigation, les formulaires, le typage, Firebase, NoSQL, les flux de données, la persistance, les erreurs, les constantes, les DTO, l'ODM, la localisation et les principes d'architecture étudiés au cours.

3.3. Évaluation de l'oral

La note tient compte de la clarté de la présentation, de la précision des explications, de la pertinence des justifications, de la maîtrise technique et de la capacité à répondre aux questions sans réciter un discours préparé.

Maîtrise insuffisante. Si l'étudiant est incapable d'expliquer son code, de justifier ses choix ou de défendre correctement son projet, une note inférieure à 10/20 peut être attribuée à la défense orale, même si l'application paraît aboutie ou fonctionne lors de la démonstration.

L'oral est strictement individuel. Les modalités pratiques, l'horaire et les éventuels supports autorisés sont communiqués sur Moodle.

4. Critères généraux de qualité

Pour le projet comme pour la défense orale, la qualité du code fait partie intégrante de l'évaluation. Le respect de ces règles ne donne pas de points supplémentaires : il constitue le niveau de qualité attendu. Leur non-respect peut entraîner une perte de points.

4.1. Structure, widgets et constantes

- respecter l'architecture enseignée et la séparation en trois projets distincts ;
- décomposer les interfaces en widgets courts, paramétrables et réutilisables, afin d'éviter les fichiers excessivement longs ;
- séparer la présentation, la logique applicative et l'accès aux données afin de réduire le couplage ;
- utiliser des noms explicites et maintenir une organisation cohérente des fichiers ;
- ne pas utiliser de « magic numbers » ou de chaînes arbitraires directement dans le code : ces valeurs doivent être remplacées par des constantes nommées, des thèmes, des configurations ou des types adaptés ;
- éviter la duplication et factoriser tout comportement ou composant réellement réutilisable.

4.2. Données typées, DTO et conversions

Toutes les données provenant de Firebase ou de la base NoSQL doivent être converties directement en objets métier typés au moyen des DTO, de l'ODM ou des convertisseurs étudiés au cours.

- ne jamais manipuler directement des données non typées ou des structures JSON dans les widgets et dans la logique applicative ;
- accéder aux propriétés sous la forme `objet.propriete` et non sous la forme `objet[« propriete »]` ;
- centraliser les conversions dans la couche prévue à cet effet ;
- éviter les appels manuels et dispersés à `toJson`, `fromJson` ou la construction directe de `Map<String, dynamic>` ;
- privilégier les DTO, l'ODM, les convertisseurs typés et, lorsque cela convient, la génération de code ;
- utiliser `StreamBuilder`, `FirebaseBuilder` ou les builders adaptés pour les flux en temps réel ; si un `FutureBuilder` est nécessaire, les données qu'il fournit doivent rester strictement typées.

Aucun contournement. Transformer manuellement des documents NoSQL en JSON dans l'interface, multiplier les `toJson/fromJson` ou accéder aux champs par des clés textuelles ne respecte pas l'approche demandée.

4.3. Fiabilité, sécurité et remise

- prévoir des stratégies de secours en l'absence de connexion et afficher des états de chargement et d'erreur compréhensibles ;
- regrouper les clés API et les informations d'identification, éviter leur dispersion dans le code et documenter les éventuelles limites de sécurité ;
- utiliser les versions stables adaptées et anticiper la maintenabilité du projet ;

- soigner l'historique Git : les messages de commit doivent permettre de comprendre les étapes du développement ;
- soumettre tous les fichiers nécessaires afin que le projet puisse être compilé sur les machines de l'école ;
- corriger toutes les erreurs et tous les avertissements visibles dans l'onglet Dart Analysis ;
- compiler et tester l'application sur un petit écran avant la remise afin de supprimer les débordements et les problèmes de mise en page.

5. Intégrité du travail

Le projet est strictement personnel. Il doit témoigner des connaissances, des compétences et de la progression de l'étudiant. Chaque étudiant doit être l'auteur de son projet et être capable d'en expliquer le code, l'architecture, le modèle de données et les choix techniques.

Tout code emprunté ou inspiré d'une source externe doit être cité et expliqué dans la documentation. Le partage, la reproduction ou l'adaptation substantielle du code d'un autre étudiant constitue un plagiat. Le plagiat entre projets est interdit, y compris lorsque seuls les noms, les données ou certains éléments visuels ont été modifiés.

L'utilisation ponctuelle d'une intelligence artificielle doit respecter les consignes du cours et être déclarée. Un projet ou une partie substantielle du projet dont le code a été généré de manière excessive par une intelligence artificielle, sans maîtrise personnelle démontrable, sera considéré comme nul dans l'évaluation.

Sanction. Toute fraude ou tentative de fraude, notamment le plagiat entre projets, la présentation d'un travail qui n'est pas personnel ou la dissimulation d'un usage excessif de l'intelligence artificielle, entraîne une note de 0/20 pour le projet.

La défense orale est strictement individuelle. Les réponses doivent être personnelles et données sans assistance extérieure. Une incapacité persistante à expliquer le travail remis peut conduire à une vérification de son authenticité.

Document de référence — année académique 2026-2027

Les échéances, les modalités pratiques de remise et les informations relatives à la défense orale sont publiées sur Moodle.